ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: FUZZING WHITE BOX TESTING CHO QUẢN LÝ DÒNG HỌ

Giảng viên: ThS. Vũ Quang Dũng

Nhóm 5:

2. Quách Hữu Nam (23012358 – K17 – KTPM(EL_2))

Lớp học phần: CSE703093-1-1-26(N02)

Hà Nội, tháng 10 năm 2026

DANH SÁCH PHÂN CÔNG

STT	Mã sinh viên	Họ và tên	Công việc
2	23012358	Quách Hữu Nam	Phần 2.5 (Bề mặt tấn công, ranh giới tin cậy) Phần 3.1, 3.2, 3.3 (Mô hình hóa, module 5 bước, cây trạng thái). Phần 3.4 (Đặc tả 10 đường đi) Phần 4.2, 4.3 (Đối chiếu cảnh báo mô phỏng, phân tích lỗi C-01) Phần 4.10 (Bảng tổng hợp mức độ bằng chứng).

2. Mô tả hệ thống và chương trình mục tiêu

2.5. Bề mặt tấn công và ranh giới tin cậy

Điểm nhận dữ liệu	Dữ liệu cần quan tâm	Thuộc tính cần bảo vệ
Đăng nhập, đăng ký	Định danh, mật khẩu, mã mời	Xác thực, chống lạm dụng, bảo vệ thông tin tài khoản
Route thành viên	:id, bộ lọc, thông tin cá nhân, quan hệ gia đình	Quyền trên bản ghi, toàn vẹn quan hệ và dữ liệu
Route tài chính	Số tiền, loại giao dịch, năm định mức	Tính hợp lệ của giá trị và quyền ghi dữ liệu
Route tư liệu sự kiện	Tệp tải lên, thông tin sự kiện	Quyền truy cập và giới hạn dữ liệu
Hàm C qua chương trình kiểm thử bổ sung	Độ dài, chỉ số, chuỗi, số lượng	An toàn bộ nhớ và tính đúng của điều kiện kiểm tra

JWT và kiểm tra vai trò là các bước cần thiết, nhưng chưa đủ để bảo đảm người dùng chỉ truy cập bản ghi thuộc dòng họ được phép. Tương tự, dữ liệu đã qua kiểm tra giao diện vẫn cần được xác thực tại backend vì yêu cầu HTTP có thể được tạo ngoài giao diện.

3. Phương pháp thực hiện

3.1. Mô hình hóa và thuộc tính an toàn

Chương trình được xem xét qua đầu vào, trạng thái trung gian, điều kiện chuyển tiếp và đường đi đến trạng thái kết thúc. Các thuộc tính cần kiểm tra gồm: độ dài sao chép không âm và không vượt kích thước nguồn/đích; chỉ số trong miền hợp lệ; con trỏ còn hợp lệ khi sử dụng; kích thước không mất dữ liệu trước khi sử dụng; phép duyệt dừng khi có chu trình; thao tác web được giới hạn đúng dòng họ.

Các thuộc tính này định hướng việc đọc code và thiết kế kiểm thử. Báo cáo không áp dụng SMT/BMC, không thực hiện symbolic execution và không tuyên bố đã chứng minh hình thức chương trình an toàn.

3.2. Một module trung tâm với năm bước xử lý

Module Module_Clan_Genealogy_Engine được thực thi bởi execute_clan_genealogy_module. Tại đầu mỗi ca, code xóa trace hiện tại và bộ đếm trạng thái/path mới; các mảng độ phủ toàn chiến dịch vẫn được giữ lại.

Bước	Vai trò	Vùng nhớ và hành vi chính
1 Parser	Kiểm tra tên và tiểu sử	name_buf[16], bio_buf[64]; kiểm tra độ dài và %x/%n
2 Heap Node	Cấp phát và phân loại vòng đời nút	g_member_table[16], malloc(sizeof(ClanMember)), nhánh giải phóng rồi cấp phát lại
3 Linker	Phân loại quan hệ	Kiểm tra chỉ số cha; kiểm tra đặc biệt ô phối ngẫu số 15
4 DFS BFS và Fund	Duyệt đồ thị và tính kích thước	g_parent_map, hàng đợi 64 số nguyên, phép thu hẹp về uint16_t
5 Audit Diff	Sao chép dữ liệu đối chiếu	old_bio_buf[24] và diff_preview[64]

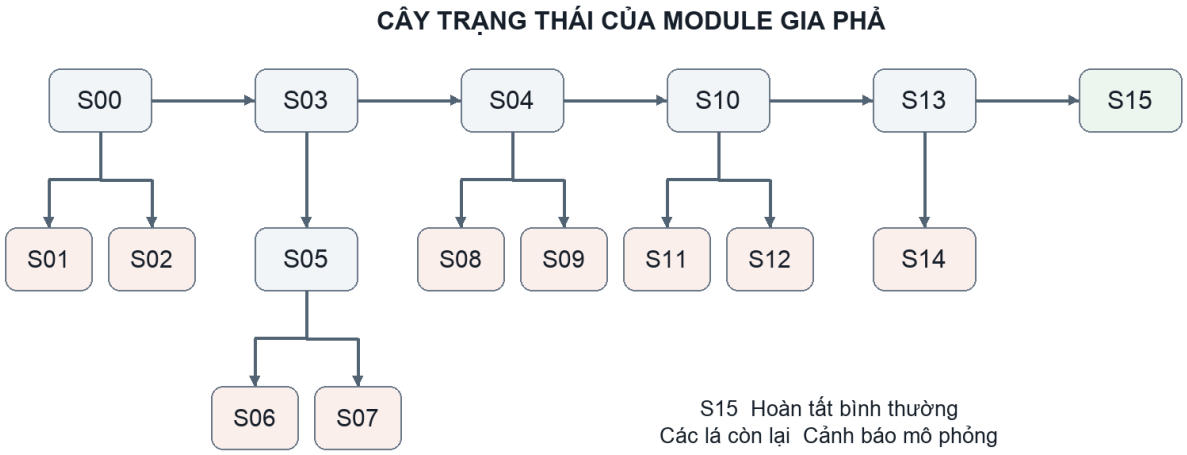
Khác với cách chạy tiếp qua các module của phiên bản trước, các nhánh bất thường hiện gọi goto finalize_path. Một ca chỉ đi tiếp nếu bước trước đạt nhánh bình thường. Sau khi kết thúc đường đi, chương trình ghi nhận trace và in độ phủ, rồi tiếp tục ca tiếp theo. “Kết thúc sớm một ca” không có nghĩa tiến trình bị crash.

3.3. Trạng thái và điều kiện thực thi

Bảng dưới đây diễn giải theo code; tên trạng thái được giữ để đối chiếu với STATE_TABLE.
Mỗi trạng thái đi kèm một nhãn mệnh đề cùng số thứ tự.

Trạng thái	Mệnh đề	Điều kiện hoặc hành động trong code
S00	P00	Vào module sau khi in đầu vào; không có phép kiểm tra NULL riêng
S01	P01	Tên dài từ 16 byte; chép tối đa 15 byte và kết thúc ca
S02	P02	Tên ngắn; tiểu sử chứa %x hoặc %n; dùng định dạng %s rồi kết thúc ca

S03	P03	Tên ngắn và tiểu sử không chứa hai mẫu %x, %n
S04	P04	Đến nhánh lifecycle_op <= 0; nút đã có hoặc vừa được cấp phát
S05	P05	lifecycle_op > 0; đã gọi free, con trỏ chưa được gán lại
S06	P06	lifecycle_op == 1; chỉ ghi nhãn UAF
S07	P07	lifecycle_op > 0 và khác 1; không chỉ riêng giá trị 2
S08	P08	Chỉ số cha ngoài 0–15; không thực hiện ghi mảng qua chỉ số đó
S09	P09	Chỉ số phối ngẫu bằng 15 và ô 15 là NULL; không giải tham chiếu NULL
S10	P10	Chỉ số cha hợp lệ và không vào nhánh kiểm tra đặc biệt ở S09
S11	P11	Ít nhất một phép duyệt vượt 32 bước rồi trả -1
S12	P12	Không có chu trình; fund_count > 16384 và số byte thu hẹp nhỏ hơn số bản ghi
S13	P13	Không vào hai nhánh cảnh báo của bước 4
S14	P14	Độ dài audit lớn hơn 24; kết thúc ca trước memcpy
S15	P15	Nhánh còn lại của audit, ghi sau memcpy; còn thiếu kiểm tra độ dài âm



Hình 3.3. Cây chuyển trạng thái của module. Các lá S01, S02, S06, S07, S08, S09, S11, S12, S14 và S15 tương ứng với 10 đường đi khai báo. S15 là lá hoàn tất bình thường; các lá khác mang nhãn cảnh báo mô phỏng.

3.4. Đặc tả 10 đường đi

Path	Chuỗi trạng thái khai báo	Ca kích hoạt
PATH_01	S00 → S03 → S04 → S10 → S13 → S15	TC01, TC02
PATH_02	S00 → S01	TC03
PATH_03	S00 → S02	TC04
PATH_04	S00 → S03 → S04 → S08	TC05
PATH_05	S00 → S03 → S04 → S09	TC06
PATH_06	S00 → S03 → S04 → S10 → S12	TC07
PATH_07	S00 → S03 → S04 → S10 → S13 → S14	TC08
PATH_08	S00 → S03 → S05 → S06	TC09
PATH_09	S00 → S03 → S05 → S07	TC10
PATH_10	S00 → S03 → S04 → S10 → S11	TC11

g_current_trace chứa tối đa tám trạng thái; đường đi dài nhất đang khai báo có sáu trạng thái. Trace lưu các mốc trong mô hình, không lưu từng bước lặp DFS/BFS, từng nhánh malloc hoặc mọi nhánh điều khiển của ngôn ngữ C. Vì vậy, 10 path là 10 chuỗi được mô hình hóa, không phải toàn bộ đường đi có thể có của chương trình.

4. Kết quả và phân tích

4.2. Cảnh báo mô phỏng và sự cố thực tế

Hàm enter_state tăng bộ đếm dựa trên chuỗi outcome_type, không dựa trên tín hiệu hệ điều hành hoặc thông báo từ sanitizer. Nhánh goto finalize_path dừng đường đi hiện tại nhưng không dừng tiến trình. Vì vậy, tám nhãn CRASH và một nhãn HANG chưa thể được gọi là tám crash thật và một lần treo thật.

Nhãn và CWE trong mô hình	Hành vi thực tế	Kết luận quan sát
S01, CWE-121	Chỉ chép 15 byte vào tên 16 byte rồi thêm ký tự kết thúc	Không tái hiện ghi tràn bộ đệm
S02, CWE-134	Dùng định dạng cố định %s với dữ liệu tiểu sử	Không dùng đầu vào làm format string
S06, CWE-416	Ghi nhãn sau free rồi cấp phát lại	Không đọc/ghi qua vùng đã giải phóng ở nhánh này
S07, CWE-415	Chỉ một lời gọi free trước khi cấp phát lại	Không thực hiện double free
S08, CWE-787	Kiểm tra chỉ số cha rồi kết thúc ca	Không ghi ngoài mảng qua chỉ số đó
S09, CWE-476	Kiểm tra ô 15 bằng NULL rồi kết thúc ca	Không giải tham chiếu con trỏ NULL
S11, CWE-674	DFS/BFS dừng sau 33 bước và trả -1	Phát hiện vượt ngưỡng, không có treo thực tế
S12, CWE-190	Thu hẹp kích thước rồi ghi nhãn; không cấp phát quỹ	Chưa có heap overflow được thực hiện
S14, CWE-125	Kết thúc ca trước memcpy khi độ dài > 24	Nhánh này không đọc vượt biên

Bảng 4.2. Bảng đối chiếu cảnh báo với thao tác thực thi. Các mã CWE ở cột đầu được lấy từ bảng mô hình, không phải danh sách lỗ hổng đã xác nhận trên ứng dụng web.

Các hàm duyệt của C dùng vòng lặp có giới hạn, không dùng đệ quy. Nhãn CWE-674 do đó không phản ánh trực tiếp cách triển khai. Nếu bỏ chặn và để chu trình gây vòng lặp không dừng, CWE-835 gần với cơ chế đó hơn [8]; hiện tại log TC11 chỉ chứng minh DFS/BFS đạt 33 bước rồi kết thúc đường đi.

4.3. Lỗi C-01 còn tồn tại với độ dài audit âm

Vị trí cập nhật: clan_genealogy_fuzzer.c, hàm execute_clan_genealogy_module, kiểm tra ở dòng 366 và memcpy ở dòng 371. Hàm audit riêng của phiên bản cũ không còn tồn tại.

```
memset(diff_preview, 0, sizeof(diff_preview));
if (tc->step5_audit_claim_len > (int)sizeof(old_bio_buf)) {
    enter_state(S14_ERR_AUDIT_OOB_READ, 14, tc);
    goto finalize_path;
}
memcpy(diff_preview, old_bio_buf,
       (size_t)tc->step5_audit_claim_len);
enter_state(S15_AUDIT_COMMITTED_OK, 15, tc);
```

Đầu vào tái hiện: ca nền có tên Nguyen Van A, tiểu sử Tieu su, slot/cha/phối ngẫu đều bằng 0, không tạo chu trình, fund_count = 10; chỉ thay step5_audit_claim_len = -1. Các bước trước audit đi theo nhánh bình thường.

Điều kiện -1 > 24 sai nên không vào S14. Giá trị âm sau đó được đổi sang size_t, trở thành số nguyên không dấu rất lớn và được dùng làm kích thước memcpy. ASan dừng tiến trình trước khi module ghi S15 hoặc hoàn tất đo path của ca này.

Bằng chứng rút gọn từ results/audit_-1.stderr.txt:

```
ERROR: AddressSanitizer: negative-size-param: (size=-1)
    ... in __asan_memcpy
    ... in execute_clan_genealogy_module
    ... in main
SUMMARY: AddressSanitizer: negative-size-param
ABORTING
```

Python ghi nhận returncode = -6, tương ứng tín hiệu 6 trên môi trường thực nghiệm. Log có cảnh báo về trình giải ký hiệu bên ngoài, nhưng vẫn nêu rõ loại lỗi và hàm liên quan. Kết quả xác nhận tham số kích thước sai và tiến trình bị dừng; không chứng minh đã ghi đè thành công bộ nhớ hay có khai thác từ mạng.

Nguyên nhân phù hợp với CWE-195, lỗi chuyển số có dấu sang số không dấu [6]. Cần ưu tiên sửa trước khi dùng hàm với đầu vào biến đổi. main gốc chưa tạo số âm và mô hình C không được nối vào API web, nên phạm vi đã xác nhận vẫn là ca bổ sung gọi module C.

4.10. Tổng hợp mức độ bằng chứng

Mã	Phát hiện	Bằng chứng	Phạm vi kết luận
C-01	Độ dài audit âm đến memcpy	ASan và mã kết thúc -6	Lỗi xác nhận trong module C
C-02	Bỏ sót count = 16384	Giá trị trước/sau thu hẹp, không có S12	Sai sót của điều kiện phát hiện
C-03	Nhãn và mô tả chưa khớp hành vi	Baseline, sizeof, các probe về mệnh đề	Tám nhãn CRASH chưa là tám crash thực
C-04	Path chỉ được nhận dạng bằng trạng thái cuối	Code và trace đơn vị S00 → S15	Giới hạn của bộ đo, không phải đường đi tự nhiên đã xác nhận
WEB-01	Truy vấn theo id thiếu phạm vi dòng họ	Route và middleware không thay đổi	Nguy cơ phân quyền cần kiểm thử tích hợp
WEB-02	runSchema bỏ các câu tạo bảng chính	Hàm gốc chạy với kết nối giả	Lỗi được tái hiện ở cấp hàm